

RELATÓRIO TÉCNICO

Otimização de Modelos de Diagnóstico via Algoritmos Genéticos + LLM

Tech Challenge — Fase 2 (Algoritmos Genéticos e LLM)
Pós-Graduação em IA para Devs — FIAP

Projeto 1 — Otimização de Modelos de Diagnóstico

Fernando Marques do Vale - rm375623

Repositório: github.com/fernando-m-vale/IA-For-Devs

Sumário

1. Introdução e Contexto do Problema	3
1.1 Projeto escolhido e justificativa	3
1.2 Modelos otimizados	3
2. Arquitetura do Sistema	4
2.1 Stack	4
3. Algoritmo Genético	5
3.1 Representação e espaço de busca	5
3.2 Operadores genéticos	5
3.3 Função fitness	6
3.4 Experimentos realizados	6
3.5 Resultados e comparativo	7
4. Monitoramento e Logging	8
5. Integração com LLM	9
5.1 Setup e decisões técnicas	9
5.2 Prompt engineering	9
5.3 Casos de diagnóstico testados	9
5.4 Avaliação de qualidade das interpretações	10
6. Discussão	11
6.1 O sistema pode ser utilizado na prática?	11
6.2 Limitações identificadas	11
7. Conclusão	12
8. Informações Complementares	13
8.1 Repositório do projeto	13
8.2 Vídeo de demonstração	13
8.3 Fonte do dataset	13
8.4 Modelo LLM	13

1. Introdução e Contexto do Problema

Este documento detalha o desenvolvimento do Tech Challenge da Fase 2 (Algoritmos Genéticos e LLM) do curso de Pós-Graduação em IA para Devs da FIAP.

Após a fase 1, onde foram construídos modelos de Machine Learning para diagnóstico de câncer de mama (Regressão Logística, Random Forest e KNN) com recall médio de até 94.4% em validação cruzada, a Fase 2 aborda dois desafios adicionais:

- Otimização dos hiperparâmetros dos modelos existentes via Algoritmos Genéticos, visando melhorar ainda mais o desempenho preditivo.
- Integração com LLM (Large Language Model) local para gerar explicações em linguagem natural dos diagnósticos, tornando os resultados interpretáveis por profissionais de saúde.

1.1 Projeto escolhido e justificativa

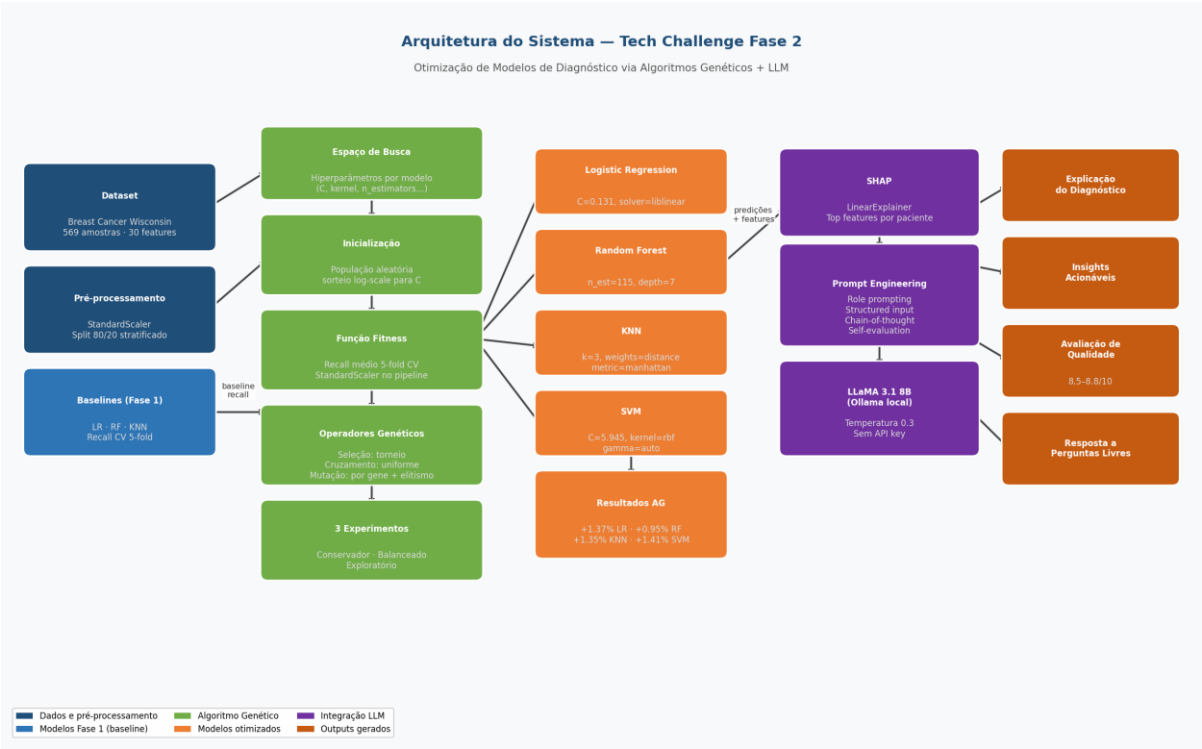
Dos projetos expostos, resolvi optar pelo Projeto 1, Otimização de Modelos de Diagnóstico, pois no meu entendimento eu poderia dar uma continuidade direta com a Fase 1, dados que os modelos já construídos e validados servem como ponto de partida para o AG, eliminando retrabalho e permitindo comparação direta de desempenho antes e depois da otimização.

1.2 Modelos otimizados

Os 3 modelos da Fase 1 (Regressão Logística, Random Forest, KNN) foram mantidos como baseline, e o SVM foi adicionado como quarto modelo, pois seu espaço de hiperparâmetros (C, kernel, gamma) torna a otimização por AG particularmente interessante e demonstra que a solução é generalizável além dos modelos originais.

2. Arquitetura do Sistema

O diagrama abaixo apresenta o fluxo completo do sistema, desde o dataset até os outputs gerados pela LLM:



O sistema opera em dois pipelines complementares:

Pipeline de otimização (AG): dataset => pré-processamento => AG (espaço de busca, fitness, operadores, 3 experimentos) => modelos otimizados com melhores hiperparâmetros.

Pipeline de interpretação (LLM): predições + valores SHAP => prompt engineering => LLaMA 3.1 8B (Ollama local) => explicações em linguagem natural, insights acionáveis e avaliação de qualidade.

2.1 Stack

Componente	Tecnologia	Justificativa
Gerenciador de ambiente	Poetry (pyproject.toml)	Reprodutibilidade total via poetry.lock
ML e AG	scikit-learn, numpy, pandas	Continuidade com Fase 1
Interpretabilidade	SHAP (LinearExplainer)	Features mais relevantes por predição
LLM local	Ollama + LLaMA 3.1 8B	Gratuito, sem API key, bom em português
Logging e visualização	logging, matplotlib, seaborn	Tracking de gerações do AG

3. Algoritmo Genético

O Algoritmo Genético (AG) é uma metaheurística de otimização inspirada na evolução biológica. Cada 'indivíduo' da população representa um conjunto de hiperparâmetros candidatos; o AG evolui essa população ao longo de gerações, favorecendo os indivíduos com maior 'fitness' (recall em validação cruzada).

3.1 Representação e espaço de busca

Cada indivíduo é representado como um dicionário Python com um valor para cada hiperparâmetro do modelo. O espaço de busca foi definido separadamente para cada modelo:

Modelo	Hiperparâmetros	Faixa / Opções
Logistic Regression	C	0.001–100 (escala log)
	solver	lbfgs, liblinear
	max_iter	100–2000
Random Forest	n_estimators	10–300
	max_depth	3–20
	min_samples_split	2–10
	min_samples_leaf	1–10
KNN	n_neighbors	1–30
	weights	uniform, distance
	metric	euclidean, manhattan
SVM	C	0.01–100 (escala log)
	kernel	rbf, poly, sigmoid
	gamma	scale, auto

Escala logarítmica para C: o parâmetro C varia em ordens de magnitude (0.001 a 100). Sorteio em escala linear concentraria valores entre 50 e 100, ignorando a faixa abaixo de 1. O sorteio em log10 garante que cada ordem de magnitude tenha a mesma probabilidade de exploração.

3.2 Operadores genéticos

Seleção por torneio: sorteia `tournament_size=3` indivíduos aleatoriamente e retorna o de maior fitness. Garante pressão seletiva sem eliminar completamente indivíduos menos aptos, mantendo diversidade genética.

Cruzamento uniforme: cada gene do filho é herdado do pai A com probabilidade `crossover_rate`, ou do pai B caso contrário. Escolhido em vez de single-point porque hiperparâmetros não têm ordem natural a preservar em blocos.

Mutação por gene: cada hiperparâmetro é substituído por valor aleatório novo (dentro do espaço de busca) com probabilidade `mutation_rate`. Permite que múltiplos parâmetros mudem simultaneamente.

Elitismo (n=2): os 2 melhores indivíduos de cada geração são preservados intactos, garantindo que o AG nunca perca a melhor solução encontrada.

3.3 Função fitness

A função fitness avalia cada indivíduo treinando o modelo com seus hiperparâmetros via validação cruzada estratificada (5-fold) e retornando o recall médio. O recall foi escolhido como métrica prioritária pela mesma razão da Fase 1: em diagnóstico médico, falso negativo (classificar maligno como benigno) tem custo clínico mais alto.

O StandardScaler é encapsulado em um Pipeline dentro de cada fold. Combinações inválidas de hiperparâmetros recebem fitness=0.0 via try/except, sendo naturalmente eliminadas pelo AG.

3.4 Experimentos realizados

Foram realizados 3 experimentos com configurações distintas do AG, conforme exigência do enunciado:

Experimento	População	Gerações	Mutação	Cruzamento
1 — Conservador	10	10	0.10	0.80
2 — Balanceado	20	20	0.20	0.70
3 — Exploratório	30	30	0.30	0.60

3.5 Resultados e comparativo

Baselines da Fase 1 (recall médio 5-fold CV com hiperparâmetros padrão):

Modelo	Baseline (Recall)
Logistic Regression	0.9440
Random Forest	0.9340
KNN	0.9250
SVM	0.9576 (calculado nesta fase)

Tabela comparativa final — melhor resultado encontrado pelo AG para cada modelo:

Modelo	Baseline	AG Otimizado	Ganho	Melhor config
Logistic Regression	0.9440	0.9577	+0.0137	Pop=20, Mut=0.2
Random Forest	0.9340	0.9435	+0.0095	Pop=20, Mut=0.2
KNN	0.9250	0.9385	+0.0135	Pop=10, Mut=0.1
SVM	0.9576	0.9717	+0.0141	Pop=20, Mut=0.2

Melhores hiperparâmetros encontrados pelo AG:

Modelo	Hiperparâmetros otimizados
Logistic Regression	C=0.131, solver=liblinear
Random Forest	n_estimators=115, max_depth=7, min_samples_split=3
KNN	n_neighbors=3, weights=distance, metric=manhattan
SVM	C=5.945, kernel=rbf, gamma=auto

Todos os 4 modelos melhoraram em relação ao baseline. O Experimento 2 (Balanceado) venceu em 3 de 4 modelos, confirmando a recomendação da literatura: populações muito pequenas convergem prematuramente (Exp 1); mutação muito alta dificulta a convergência (Exp 3). O KNN foi exceção, seu espaço de hiperparâmetros pequeno (3 parâmetros) é explorado adequadamente mesmo com população pequena. O Random Forest foi o mais custoso computacionalmente (~20 minutos no Exp 3), pois cada avaliação de fitness envolve treinar uma floresta completa 5 vezes.

4. Monitoramento e Logging

O módulo `src/logging_monitor.py` implementa o monitoramento estruturado do AG, conforme requisito de 'logging adequado para tracking de desempenho':

- Logging estruturado com timestamp, nível e mensagem formatados, emitido a cada geração, mostrando melhor fitness, fitness médio e pior fitness da população.
- Gráficos de evolução do fitness por geração (melhor, médio e pior), com área sombreada mostrando a amplitude da população, um indicador visual de convergência e diversidade genética.
- Gráfico comparativo baseline vs. AG otimizado para todos os modelos, com valores numéricos anotados nas barras.
- Tabela resumo por experimento exportável como DataFrame pandas, com baseline, melhor resultado, ganho absoluto e tempo de execução.
- Serialização dos resultados de cada experimento em JSON (`outputs/results/`), possibilitando recarregar e analisar resultados sem reexecutar o AG.

5. Integração com LLM

5.1 Setup e decisões técnicas

Foi utilizado o modelo LLaMA 3.1 8B via Ollama, rodando localmente sem necessidade de API key ou conexão à internet durante o uso. Escolhi essa opção pela gratuidade (sem custo de API), privacidade dos dados médicos (processamento 100% local) e boa qualidade de geração em português.

Decisão	Escolha	Justificativa
Modelo LLM	LLaMA 3.1 8B	Melhor qualidade disponível para 16GB RAM
Temperatura (explicações)	0.3	Consistência > criatividade em contexto médico
Temperatura (avaliação)	0.1	JSON estruturado exige mínima variabilidade
Versão SHAP	$\geq 0.44, < 0.52$	Compatibilidade com Python ≥ 3.10

5.2 Prompt engineering

Trabalhei na implementação de 5 técnicas no prompt engineering:

- 1. Role prompting:** system prompt define o papel ('assistente de apoio ao diagnóstico em oncologia'), restrições explícitas ('nunca emitir diagnóstico definitivo', 'sempre reforçar papel do médico') e formato esperado da resposta.
- 2. Structured input:** dados do diagnóstico formatados de forma consistente, predição, confiança do modelo, features mais relevantes com valores SHAP e direção do impacto.
- 3. Output format specification:** instrução explícita da estrutura esperada: resumo do resultado, interpretação das características mais relevantes, limitações e recomendação.
- 4. Chain-of-thought:** prompt de insights acionáveis solicita raciocínio passo a passo: implicação biológica, atenção prioritária, exames complementares, conduta sugerida.
- 5. Self-evaluation prompting:** a própria LLM avalia sua resposta anterior segundo 4 critérios (precisão técnica, clareza, segurança, utilidade clínica), retornando JSON estruturado com scores 0–10 e justificativa.

5.3 Casos de diagnóstico testados

Foram selecionados 3 casos representativos do conjunto de teste para demonstração:

Caso	Paciente	Predição	Prob. Maligno	Real	Feature top-1
A	TEST-108	Maligno	1.000	Maligno	area3
B	TEST-091	Benigno	0.000	Benigno	fractal_dimension2
C	TEST-112	Benigno	0.471	Maligno	texture3

Caso C — o mais relevante clinicamente: o modelo previu 'Benigno' com probabilidade 47.1%, mas o diagnóstico real era Maligno (falso negativo). A LLM identificou sinais contraditórios nas features (texture sugere benigno, symmetry3 sugere maligno), comunicando adequadamente a incerteza e reforçando a necessidade de revisão médica cuidadosa em casos limítrofes.

5.4 Avaliação de qualidade das interpretações

Caso	Precisão técnica	Clareza	Segurança	Utilidade clínica	Média
A - Maligno	9	8	9	9	8.8
B - Benigno	9	8	9	8	8.5
C - Limítrofe	9	8	9	8	8.5

Notas consistentemente altas (8.5 a 8.8/10) nos 3 casos. Destaque para o critério 'Segurança' (9/10 em todos), indicando que a LLM respeitou adequadamente os limites do system prompt e nunca emitiu diagnóstico definitivo e sempre reforçou o papel do médico.

Limitação de tempo: 294 a 474 segundos por chamada em CPU. Em produção com GPU ou API de nuvem, o tempo certamente cairia. Esta limitação é diretamente ligada ao meu ambiente de desenvolvimento, não do design da solução.

6. Discussão

6.1 O sistema pode ser utilizado na prática?

Sim, mas com ressalvas importantes. O pipeline AG + LLM demonstrou capacidade de:

- Melhorar o recall de todos os modelos avaliados (ganhos de +0.95% a +1.41%), com o SVM atingindo 97.17% de recall após otimização.
- Gerar explicações em linguagem natural coerentes com os dados, úteis para profissionais de saúde que precisam entender o raciocínio do modelo.
- Identificar e comunicar incerteza em casos limítrofes (Caso C), que é exatamente onde o apoio clínico é mais necessário.

6.2 Limitações identificadas

- O AG é estocástico, resultados podem variar entre execuções com seeds diferentes. `random_state=42` garante reprodutibilidade desta execução específica.
- O tempo de otimização do Random Forest (~20 minutos no Exp 3) é alto para uso iterativo em produção. Pré-computar e cachear os melhores hiperparâmetros é o caminho recomendado. Lembrando do limite de infra.
- A avaliação de qualidade da LLM é auto-referencial, uma avaliação humana por especialistas médicos seria mais robusta.
- O modelo de ML, mesmo otimizado, ainda comete falsos negativos (Caso C é um exemplo real do conjunto de teste), o médico deve sempre ter a palavra final.

7. Conclusão

Pude observar nesse projeto como Algoritmos Genéticos e LLMs podem complementar o pipeline de Machine Learning desenvolvido na Fase 1 de duas formas distintos:

Eixo preditivo (AG): o AG encontrou hiperparâmetros superiores aos padrões em 100% dos modelos avaliados, com ganhos de recall de até +1.41%. O Experimento 2 (configuração Balanceada) apresentou o melhor desempenho em 3 de 4 modelos, confirmando recomendações da literatura sobre tamanho de população e taxa de mutação.

Eixo interpretativo (LLM): a integração com LLaMA 3.1 8B via Ollama produziu explicações em linguagem natural com notas de qualidade consistentes de 8.5 a 8.8/10, respeitando as restrições de segurança do system prompt e identificando adequadamente casos de alta incerteza.

A escolha de LLM local (Ollama) foi especialmente adequada para o contexto médico, onde dados de pacientes não devem ser enviados para servidores externos.

8. Informações Complementares

8.1 Repositório do projeto

<https://github.com/fernando-m-vale/IA-For-Devs/tree/main/fase-02>

8.2 Vídeo de demonstração

<https://www.youtube.com/watch?v=5fyzxE77Jc8>

8.3 Fonte do dataset

Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1995). Breast Cancer Wisconsin (Diagnostic) [Dataset]. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

8.4 Modelo LLM

Meta AI. (2024). LLaMA 3.1 — Large Language Model. Disponível via Ollama:
<https://ollama.com/library/llama3.1>